

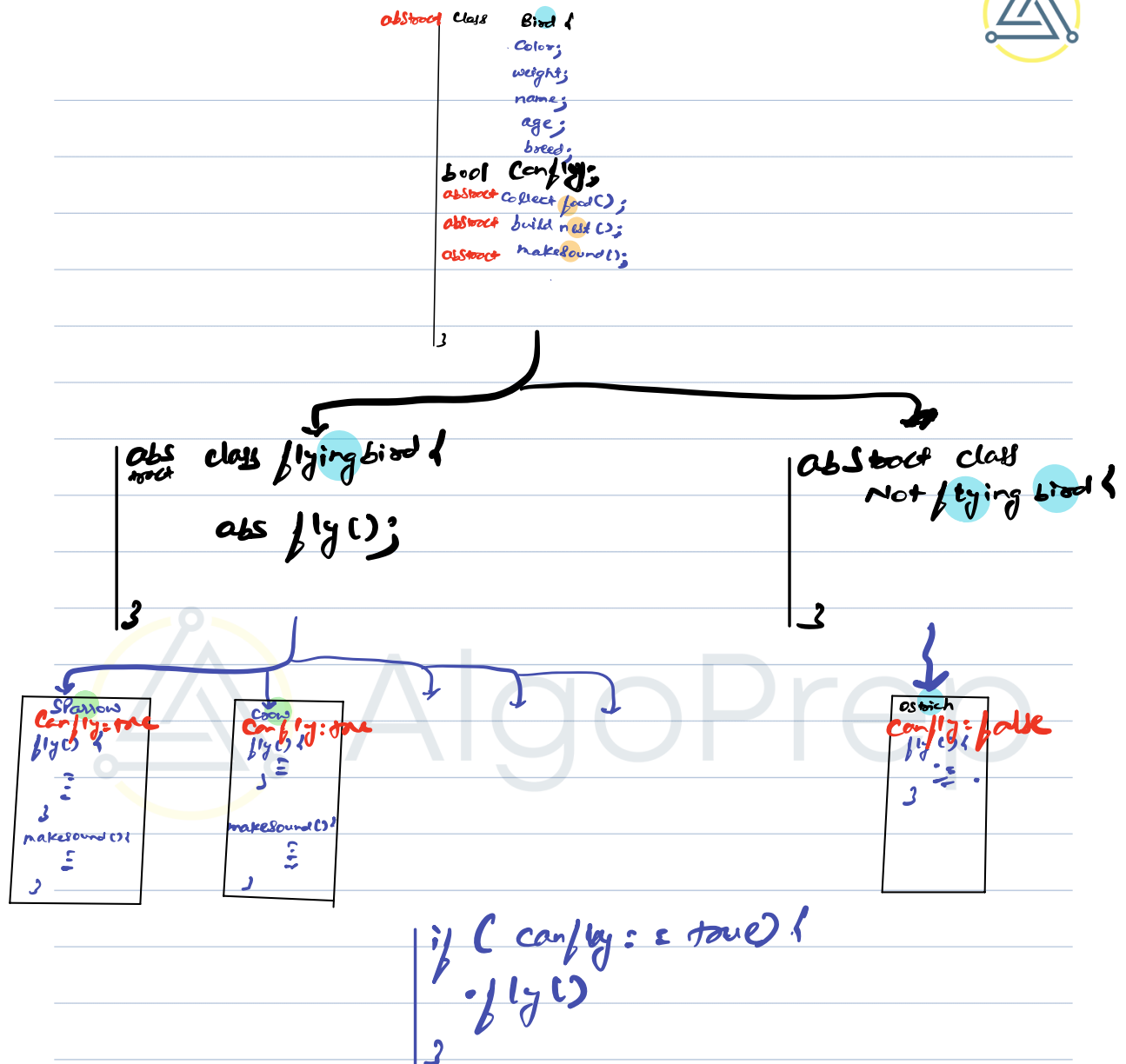


Today's Agenda

↳ SOLID Principle



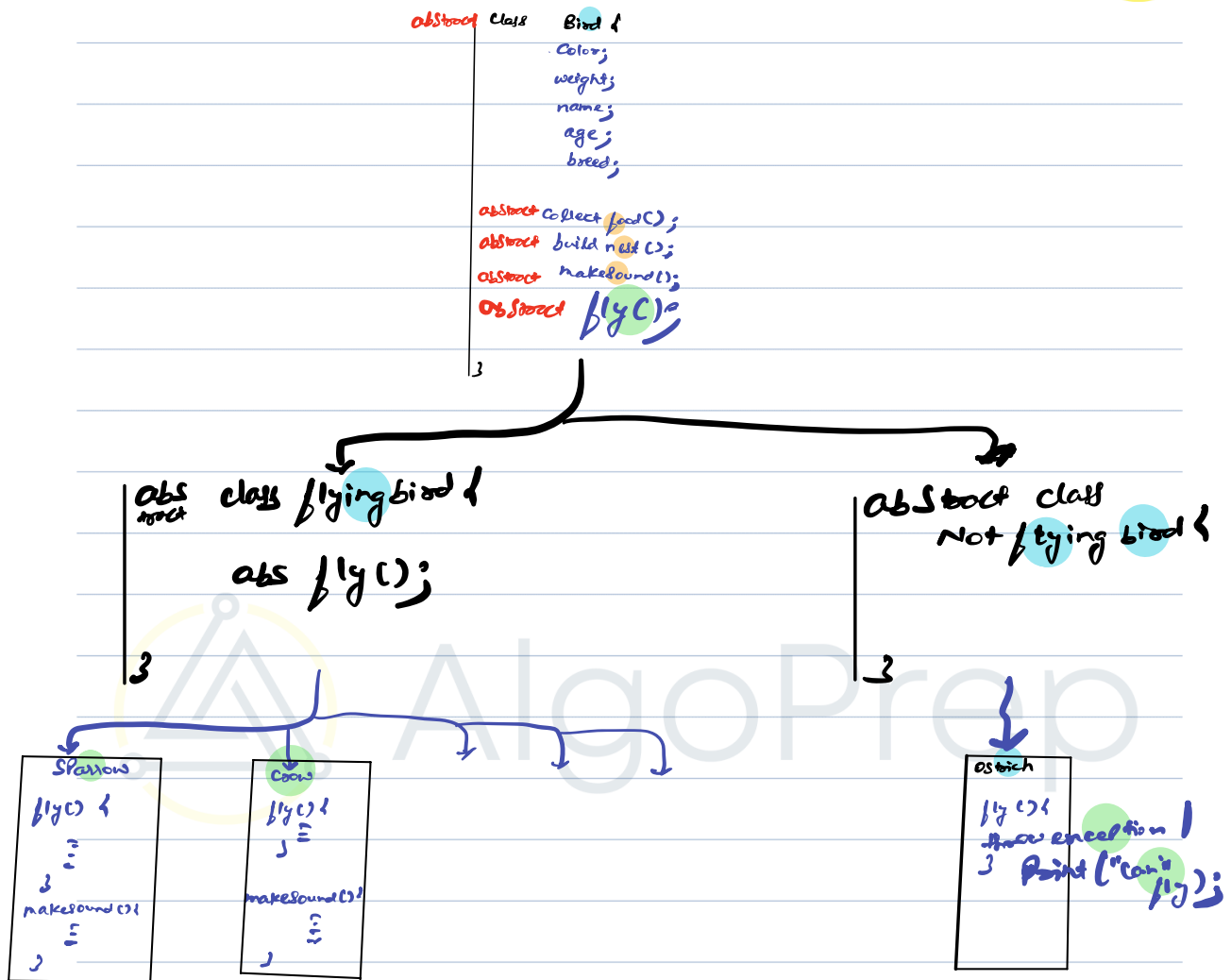
AlgoPrep



→ client is using this code.

↳ written in documentation.

↓
fly()



fly()



L → Liskov's Substitution Principle

↳ child classes should do exactly what their Parents class expects them to do.

newspaper
↳ Paper Printing

abs Printer {

Print something
on paper ←

abs Print();

m.c();
m.ec();

3

XX
inherit

you want to do
commandline Printing

class CommandLinePrinting {

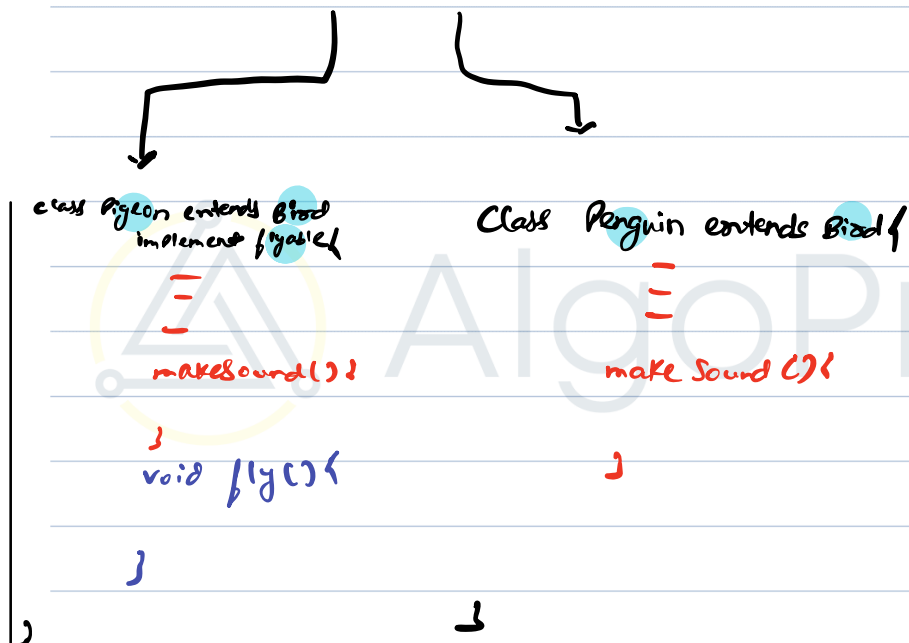
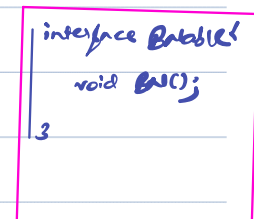
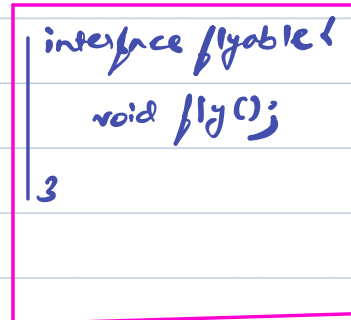
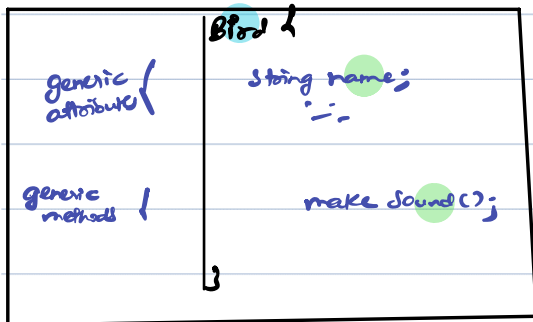
Print() {

3

}



v4 → creating interface for optional method.



Requirements

All the birds that can fly can BN and birds that can't fly can't BN.

build nest



↳ Can we combine the 2 interfaces we took in last solution??

↳ yes

```
abstract class Bird {  
    color;  
    weight;  
    name;  
    age;  
    breed;  
  
    abstract collect food();  
    abstract makeSound();  
}
```

```
interface flyable {  
    void fly();  
    void enable();  
}
```

lots of birds

```
class crow {  
    extends Bird implements  
    flyable  
    void fly() {  
    }  
}
```

↳ if you found a bird which can fly but can't BN.



* Interface Segregation Principle.

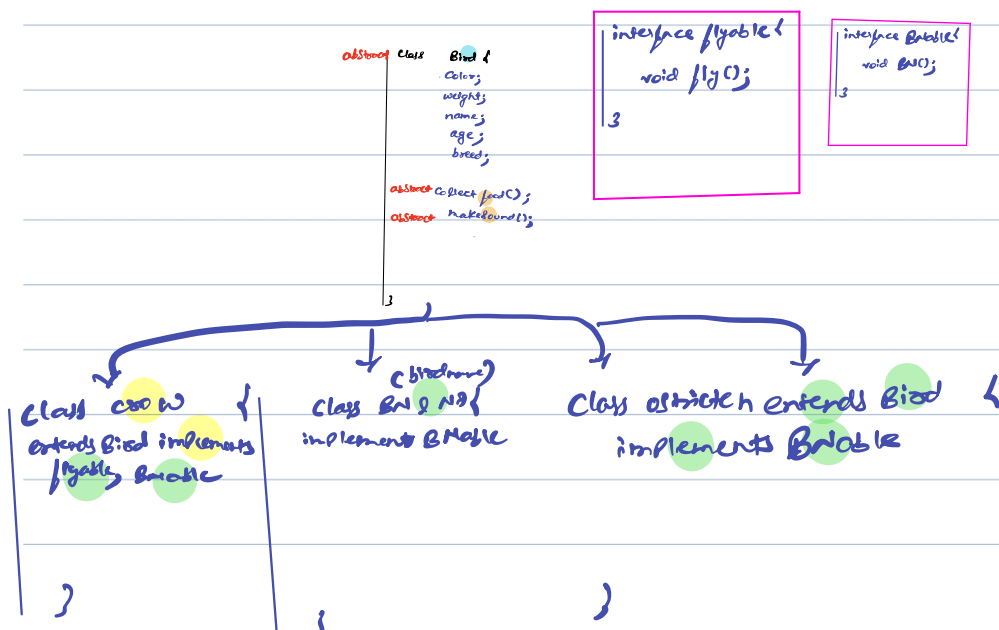
↳ interfaces should be as light as possible.

✓
of methods in interface should be minimum possible. {ideally 1}

↓
if interface has more than 1 method, they should be related to each other.

→ interface segregation principle is equivalent to SRP for interfaces.

Correct Solution even with
↳ considering above requirements.





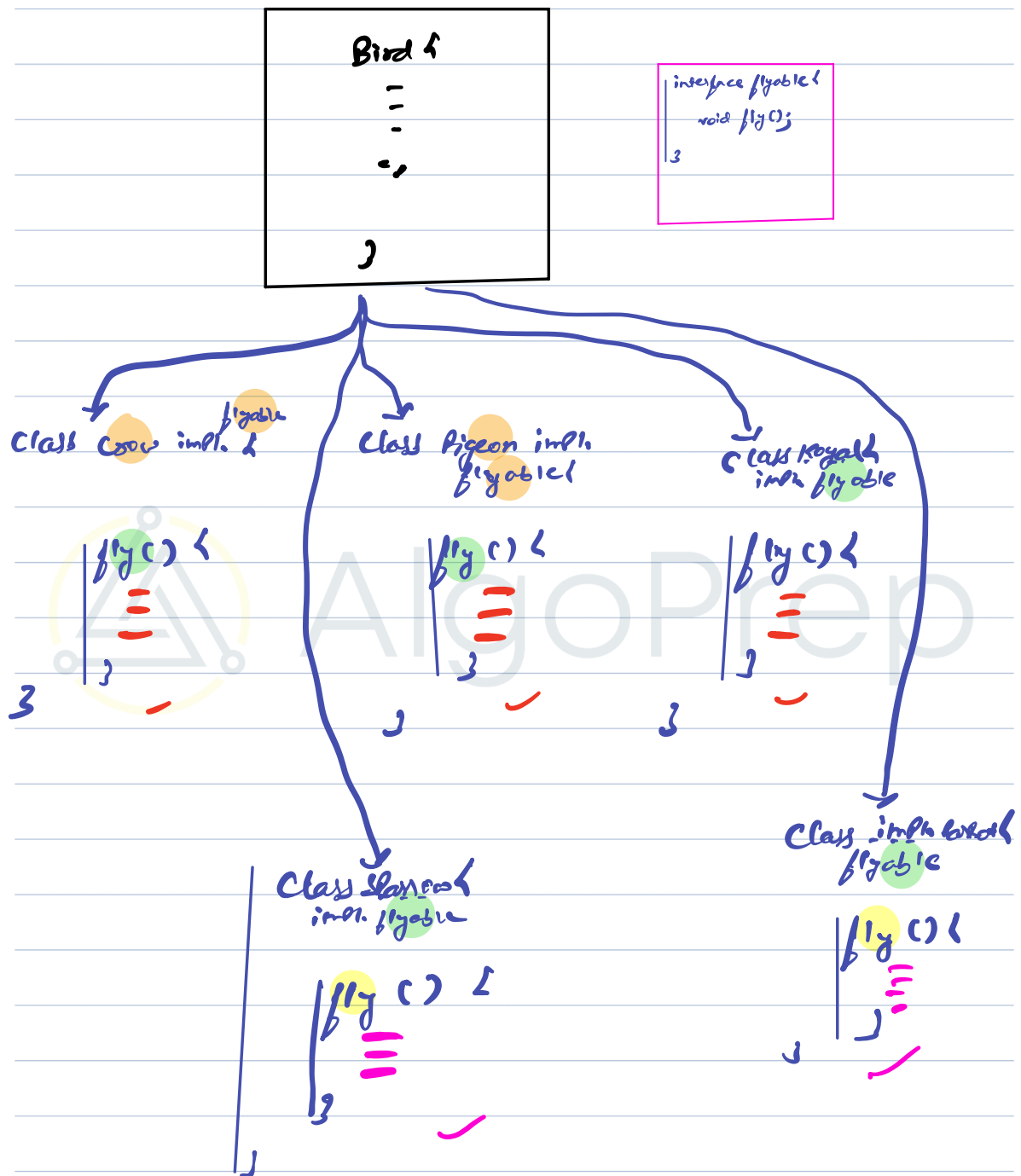
example with interface having more than 1 method:



```
Interface Stack {  
  
    Push();  
    Pop();  
    Size();  
    top();  
}
```

→ functional interface: interfaces with only 1 method.

Break till 9:25 PM



→ DRY {code duplication}

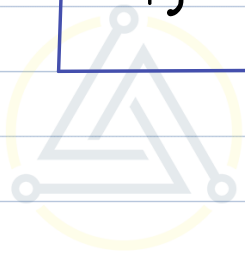
within same
class



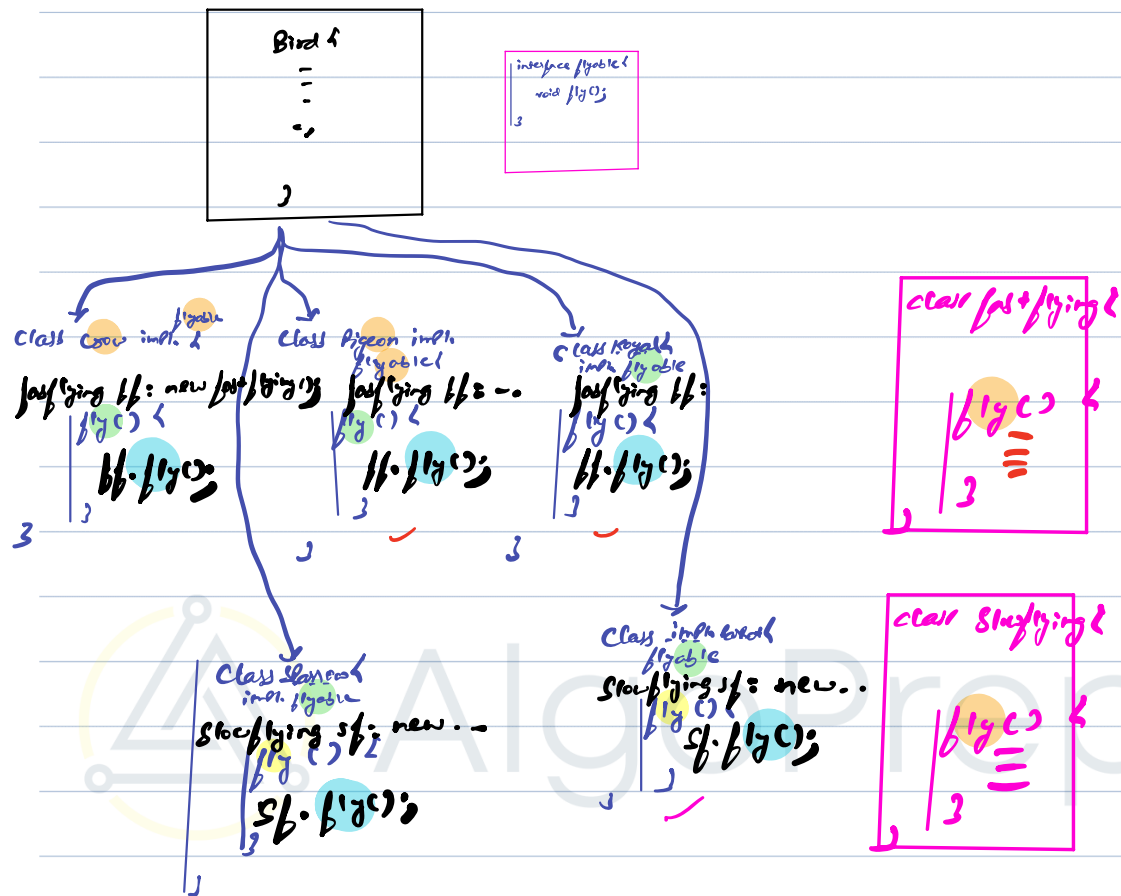
```
Code 1() {  
    //  
    //  
    //  
}  
  
Code 2() {  
    //  
    //  
    //  
}
```



```
Code 3() {  
    //  
    // { common  
    // code }  
    //  
}  
  
Code 1() {  
    Code 3();  
}  
  
Code 2() {  
    Code 3();  
}
```



AlgoPrep

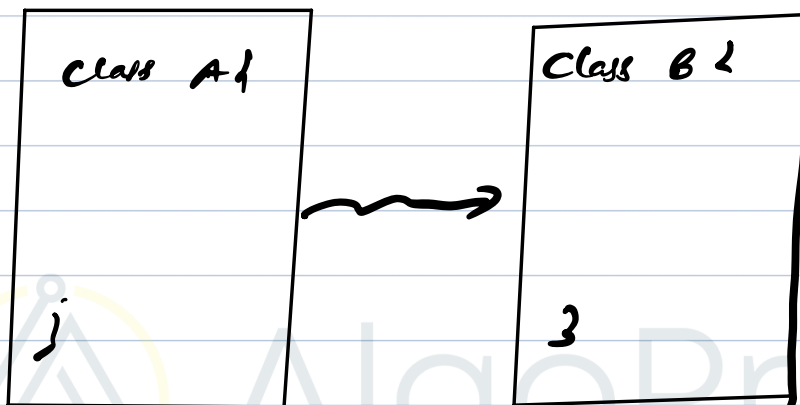


not a good practice.



* D: **Dependency Inversion Principle.**

↳ No 2 class should ideally depend on each other. They should depend via an interface.



↳ not ideal.

yesBank();



→ Paytm → ~~Paytm~~ ~~Payments~~ ~~bank~~
↓
PaytmPaymentsBank PPB = new
PaytmPaymentsBank();
↓
PPB.transferMoney();
PPB.fetchBalance();

```
class PaytmPaymentsBank {  
    - transferMoney();  
    - fetchBalance();  
    ...  
}
```

↓
yesBank yb = new yesBank();
.
yb.transferMoney();
yb.fetchBalance();
}

include
→ Dependency inversion principle given by RBI

```
interface Bank {  
    transferMoney();  
    fetchBalance();  
}
```

yesBank() ↑

→ Bank b = new ~~PaytmPaymentsBank~~();
↳ b.transferMoney();
↳ b.fetchBalance();

→ Bank b = new ~~PaytmPaymentsBank~~();
new yesBank();

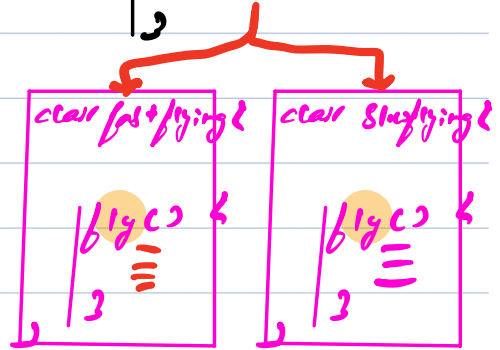
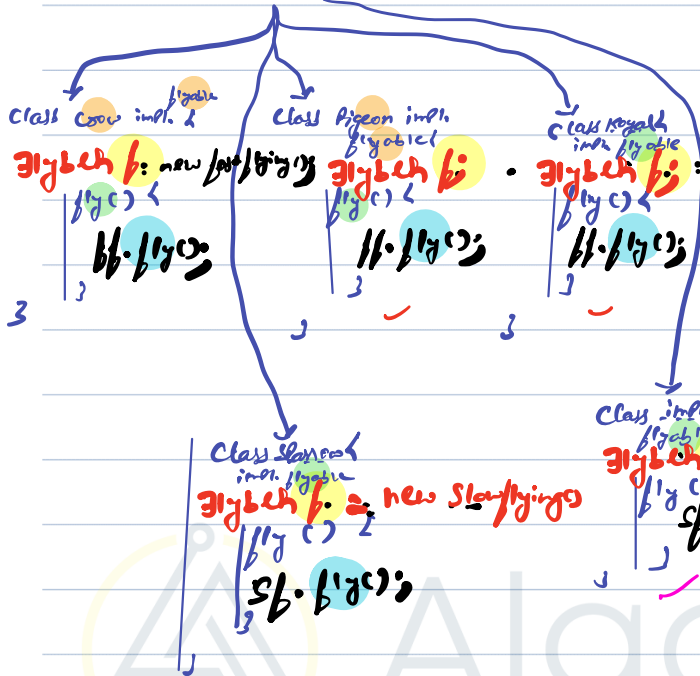
following
dependency
inversion.

```
Bird {  
    ...  
}
```

```
interface flyable {  
    void fly();  
}
```



```
interface flyser {  
    void fly();  
}
```



AlgoPrep